Building a Simple Agent Framework 4

We've discussed Goals, Actions, Memory, and Environment, but there's another crucial component we need to explore: the AgentLanguage. This component serves as the translator between our structured agent components and the language model's input/output format. Think of it as a diplomatic interpreter that ensures clear communication between two different worlds: our agent's structured OOARR components and the LLM's text-based interface.

As we have already seen, there are multiple ways that we can prompt the LLM for a next action. For example, we can have the LLM generate a standard completion with text that we parse or use function calling to extract an action. There are also many different ways that we could represent memories to the LLM, from concatenating them into a single string to including them as individual message entries in ChatML. The AgentLanguage allows us to create reusable strategies for handling these concerns and plug them into the agent.

For example, we might define an AgentLanguage that always constructs a system message explaining the agent's role, followed by a user message containing the agent's current observations, memory, and a request for the next action. Alternatively, we could use function calling to directly extract structured actions, bypassing the need for parsing. Each of these choices influences how the LLM reasons and responds, shaping the agent's behavior.

The Role of AgentLanguage

The AgentLanguage component has two primary responsibilities:

1. **Prompt Construction**: Transforming our OOARR components into a format the LLM can understand
2. **Response Parsing**: Interpreting the LLM's response to determine what action the agent should take

Let's look at how this works in practice, starting with the base abstract class:

```
class AgentLanguage:
    def construct_prompt(self,
                         actions: list[Action],
                         environment: Environment,
                         goals: list[Goal],
                         memory: Memory) -> Prompt:
        raise NotImplementedError("Subclasses must implement this method")

    def parse_response(self, response: str) -> dict:
        raise NotImplementedError("Subclasses must implement this method")
```

This abstract class defines the interface that all agent languages must implement. Let's examine three different implementations to understand how we can adapt our agent's communication style.

Where this Fits in the Agent Loop

Let's examine how the AgentLanguage component integrates with each stage of the loop, transforming raw data into meaningful communication and back again.

Consider this portion of our agent's run method:

```
def run(self, user_input: str, memory=None, max_iterations: int = 10) -> Memory:
    memory = memory or Memory()
    self.add_current_task(memory, user_input)

    for _ in range(max_iterations):
        # 1. Build prompt using AgentLanguage
        prompt = self.construct_prompt(self.goals, memory, self.actions)

        # 2. Get LLM response
        response = self.prompt_llm_for_action(prompt)

        # 3. Parse response using AgentLanguage
        action, invocation = self.get_action(response)

        # 4. Execute action in environment
        result = self.environment.execute_action(action, invocation["args"])

        # 5. Update memory
        self.update_memory(memory, response, result)

        if self.should_terminate(response):
            break

    return memory
```

At two crucial points in this loop, the AgentLanguage acts as an interpreter between our structured world and the LLM's text-based world:

Stage 1: Constructing the Prompt

When the agent needs to decide its next action, the AgentLanguage takes our OOARR components and transforms them into a format the LLM can understand. This transformation involves several steps:

```
def construct_prompt(self, goals: list[Goal], memory: Memory, actions: ActionRegistry):
    # The AgentLanguage decides how to present each component to the LLM
    prompt = []

    # Transform goals into instructions
    prompt += self.format_goals(goals)

    # Transform available actions into tool descriptions
    prompt += self.format_actions(actions.get_actions())

    # Transform memory into conversation context
    prompt += self.format_memory(memory)

    return Prompt(messages=prompt, tools=tools)
```

For example, when using function-calling, this might produce:

```
{
    "messages": [
        {"role": "system", "content": "Your goal is to process all files..."},
        {"role": "user", "content": "I've uploaded the files..."},
        {"role": "assistant", "content": "I'll start by reading the file..."}
    ],
    "tools": [
        {
            "type": "function",
            "function": {
                "name": "read_file",
                "description": "Reads a file from the system",
                "parameters": {
                    "type": "object",
                    "properties": {
                        "file_path": {"type": "string"}
                    }
                }
            }
        }
    ]
}
```

Stage 2: Parsing the Response

After the LLM generates a response, the AgentLanguage must interpret it to determine what action the agent should take:

```
def get_action(self, response):
    # AgentLanguage parses the LLM's response into a structured format
    invocation = self.agent_language.parse_response(response)

    # The parsed response is used to look up the actual action
    action = self.actions.get_action(invocation["tool"])
    return action, invocation
```

For instance, when using JSON action format, the AgentLanguage might receive this response from the LLM that mixes the agent's chatty response with a markdown block containing the specification for the action:

```
Let me analyze the contents of the file.

```action
{
 "tool": "read_file",
 "args": {
 "file_path": "file.txt"
 }
}
```
```

The AgentLanguage would then parse this to extract the JSON and convert it into a structured action:

```
{
    "tool": "read_file",
    "args": {
        "file_path": "file.txt"
    }
}
```

The AgentLanguage ensures that regardless of how the LLM prefers to communicate (function calling, JSON blocks, or natural language), the agent's core loop remains unchanged. It's like having different translators for different languages - the meaning stays the same, but the way it's expressed adapts to the audience.

Two Example Agent Languages

Let's look at two example implementations of the AgentLanguage component, each with a different approach to prompting and parsing. The first is a simple natural language approach, like what we used in our very first agents. The second is a more structured approach that leverages LLM function calling.

JSON Action Language

This language allows the LLM to output text and specify actions in special ```action markdown blocks. This is similar to what we did in our first agent examples:

```
class AgentJsonActionLanguage(AgentLanguage):
    action_format = """
Always end think step by step. Insert your thoughts here.

```action
{
 "tool": "tool_name",
 "args": {...fill in arguments...}
}
```"""

    def format_actions(self, actions: list[Action]) -> list:
        # Convert actions to a description the LLM can understand
        action_descriptions = [
            {
                "name": action.name,
                "description": action.description,
                "args": action.parameters
            }
            for action in actions
        ]

        return [{
            "role": "system",
            "content": f"""
Available Tools: {json.dumps(action_descriptions, indent=2)}

{self.action_format}
"""
        }]

    def parse_response(self, response: str) -> dict:
        """Extract and parse the action block"""
        try:
            start_marker = "```action"
            end_marker = "```"

            stripped_response = response.strip()
            start_index = stripped_response.find(start_marker)
            end_index = stripped_response.rfind(end_marker)
            json_str = stripped_response[
                start_index + len(start_marker):end_index
            ].strip()

            return json.loads(json_str)
        except Exception as e:
            print(f"Failed to parse response: {str(e)}")
            raise e
```

Function Calling Language

This next language uses the LLM's function-calling capabilities to directly specify actions. This approach helps alleviate the burden of parsing free-form text. The downside is that we don't necessarily get to see the LLM's reasoning, but the upside is that it simplifies getting valid JSON as output:

```
class AgentFunctionCallingActionLanguage(AgentLanguage):
    def format_actions(self, actions: list[Action]) -> list:
        """Convert actions to function descriptions"""
        return [
            {
                "type": "function",
                "function": {
                    "name": action.name,
                    "description": action.description[:1024],
                    "parameters": action.parameters,
                },
            }
            for action in actions
        ]

    def construct_prompt(self,
                         actions: list[Action],
                         environment: Environment,
                         goals: list[Goal],
                         memory: Memory) -> Prompt:
        prompt = []
        prompt += self.format_goals(goals)
        prompt += self.format_memory(memory)

        tools = self.format_actions(actions)

        return Prompt(messages=prompt, tools=tools)

    def parse_response(self, response: str) -> dict:
        """Parse the function call response"""
        try:
            return json.loads(response)
        except Exception as e:
            return {
                "tool": "terminate",
                "args": {"message": response}
            }
```

The Power of Swappable Languages

The ability to swap agent languages gives us remarkable flexibility in how our agent communicates. Consider these scenarios:

```
# Create an agent that uses natural language for simple tasks
simple_agent = Agent(
    goals=goals,
    agent_language=AgentJsonActionLanguage(),
    action_registry=registry,
    generate_response=llm.generate,
    environment=env
)

# Create an agent that uses function calling for complex tasks
complex_agent = Agent(
    goals=goals,
    agent_language=AgentFunctionCallingActionLanguage(),
    action_registry=registry,
    generate_response=llm.generate,
    environment=env
)
```

The same agent can behave differently just by changing its language implementation. This separation of concerns means we can:

1. Experiment with different prompt formats without changing the agent's logic
2. Support different LLM providers with their own communication styles, allowing us to adjust prompting style to match the LLM's strengths.
3. Add new response formats without modifying existing code
4. Handle errors and retry logic at the language level

Wraping Up

The AgentLanguage component is crucial because it:

1. **Centralizes Communication Logic**: All prompt construction and response parsing is in one place
2. **Enables Experimentation**: We can try different prompt strategies by creating new language implementations
3. **Improves Reliability**: Structured response formats and error handling make the agent more robust
4. **Supports Evolution**: As LLM capabilities change, we can adapt our communication approach without changing the agent's core logic

By separating the "how to communicate" from the "what to do," we create agents that can evolve and improve their interaction with LLMs while maintaining their core functionality. This flexibility is essential as language model capabilities continue to advance and new communication patterns emerge.